Lesson for the course "Human-Computer Interaction"

Learning

Prof. Luca Turchet

UNIVERSITY
OF TRENTO - Italy

Learning

- **Learning is closely connected with memory**
  - It involves the accumulation of skills and knowledge that would be impossible to achieve without memory
  - Likewise, people would not be able to remember things unless they have learned them
  - All topics we saw in the previous lesson about memory are relevant also for learning



Performing learned actions is easy

- Routine, well learned behaviors become "**automatic**"
  - Researchers have determined that performing this type of action consumes few or no conscious cognitive resources – that is, it is **not subject to the limits of attention and short-term memory**
  - Automatic activities can even be done in parallel with other activities

- How does an activity become automatic?
  - **Practice, practice, practice**



Performing novel actions is hard

- When a person first tries to do a new activity, every part of the activity requires conscious attention
  - E.g., drive a car for the first time
  - After a lot of practice all actions involved in driving a car become automatic
- **Designers of interactive systems should make the task they support faster, easier, and less error-prone**
  - **In this way the user can rely on automatic actions**

Problem solving and calculations are hard

- When facing a problem you need to think and this **takes cognitive resources**
  - The problem is not handled automatically
  - When solving problems, people often use external memory aids
    - Such tools augment our limited short-term memory and our limited ability to imagine manipulating problem elements

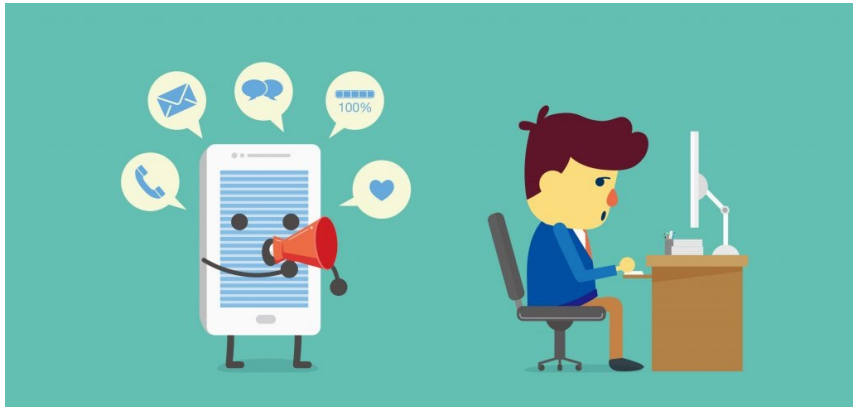


Problem solving and calculations are hard

- When using some interactive systems, certain tasks may require **training** that many users may not have, e.g., technical problem
- Example: you play a YouTube video on your computer, but there is no sound even though you can see people speaking
  - Is the problem in the video, the video player, the computer, the speaker cables, or the speakers?
  - Requires devising and executing a series of diagnostic tests that successively narrow the possible causes of the problem, which requires computer and electronics domain knowledge

Implications for UI design

- People have their own goals
  - They are using a computer to help them achieve a goal
  - They want and need to focus their attention on that goal
- Interactive systems – and designers of them – should respect that and **not distract users** by imposing technical problems and goals that users don't want
- **Interactive systems should minimize the amount of attention** users must devote to operating them, because that pulls precious cognitive resources away from the task



Guidelines

- **Prominently indicate system status and users' progress toward their goal**
  - If users can always check their status easily by direct perception, using the system will not strain their attention and short-term memory
- **Guide users toward their goals**
  - Designers can do this
    - **implicitly**, by making sure every choice-point provides a clear information "scent" that leads user toward their goal
    - or **explicitly**, by using a wizard (multistep dialog box)
  - Don't just display a bunch of options that appear equally probable and expect users to know how to start and get to their goal, especially if they won't perform the task very often

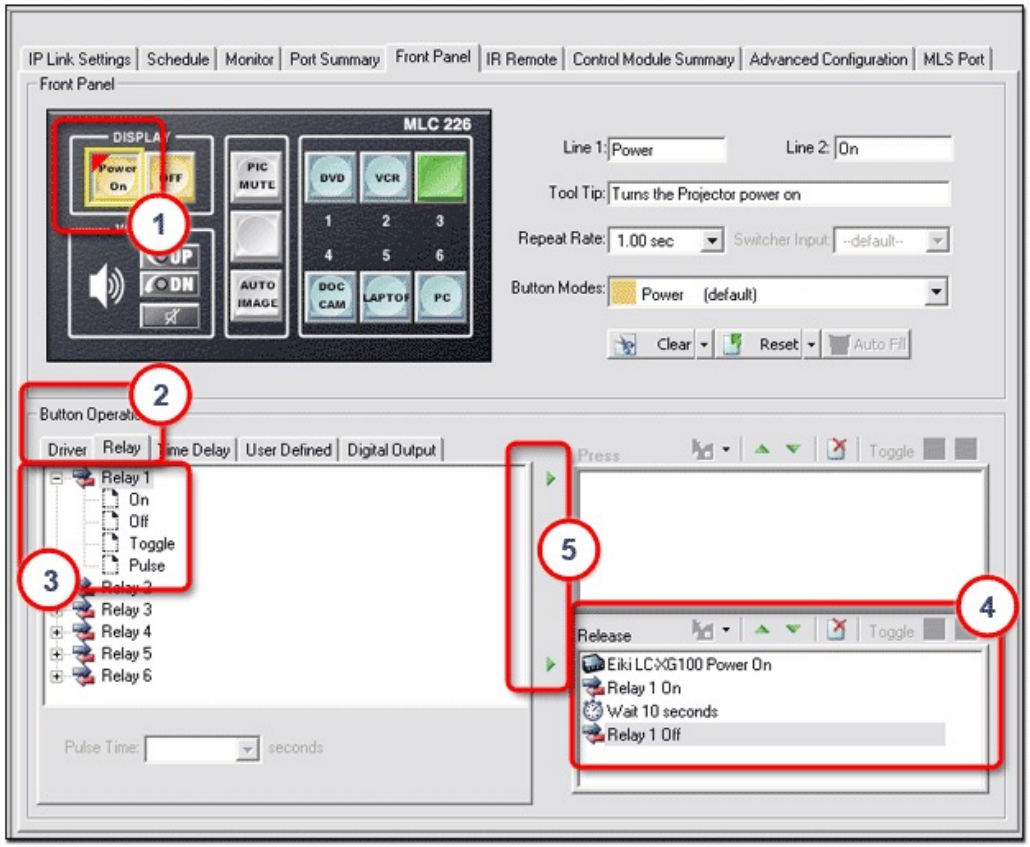
Guidelines

- **Tell users explicitly and exactly what they need to know**
  - Don't expect them to deduce information
  - Don't require them to figure things out by a process of elimination
- **Don't make users diagnose system problems**
  - E.g., a faulty network connection: such diagnosis requires technical training, which most users don't have



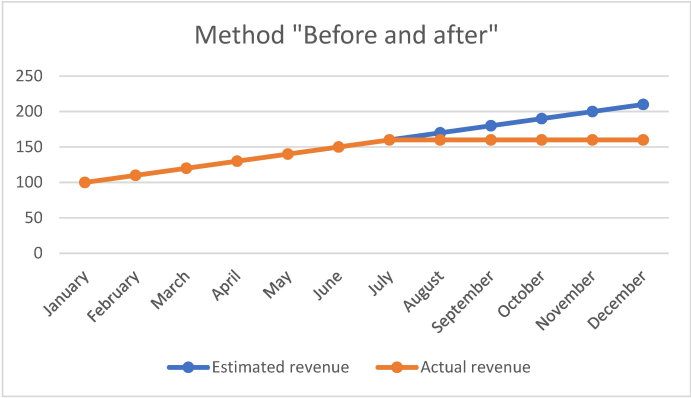
Guidelines

- **Minimize the number and complexity of settings**
  - Don't expect people to optimize combinations of many interacting settings or parameters
  - People are really bad at that

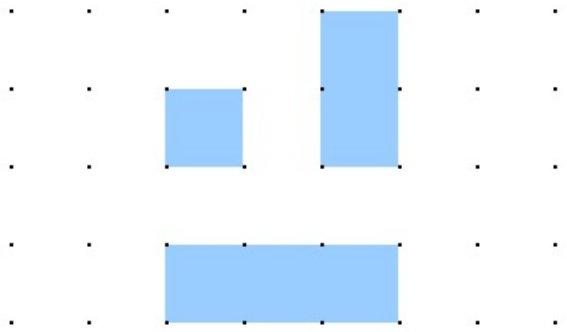


Guidelines

- **Let people use perception rather than calculation**
  - Some problems that might seem to require calculation can be represented graphically, allowing people to achieve their goals with quick perceptual estimates instead of calculation



  - E.g., in drawing tools, snap-to grids and alignment guides eliminate the need for users to determine, match, and compute coordinates of existing graphic elements when adding new ones



Guidelines

- **Make the system familiar**
  - Use concepts, terminology, and graphics that users already know to make the system as familiar to them as possible, requiring them to think about it less
  - Designers can use this approach to a certain extent even if the system provides functionality that users have not seen before
    - One way to do it is to **follow industry conventions and standards**
    - A second way is to make new software applications work like older ones that users have used before
    - A third approach is to base the design on metaphors (e.g., the desktop metaphor)
    - Finally, designers can study users to learn what is and is not familiar to them

Guidelines

- **Let the computer do the math**
  - Don't make people calculate things the computer can calculate itself



33. List the names of **all of the employers** you worked for in the last 18 months, the dates you worked for each employer, the wages you earned from each, and how you were paid. Please also indicate the employer you worked for longest by selecting the radio button next to that employer. Help

Employer Name Help	From Date (mm/dd/yyyy)	To Date (mm/dd/yyyy)	Earnings	How Paid
○				
○				
○				
○				
○				
○				
○				
○				

34. Regarding the employer in question 33 that you indicated you worked for the longest, please answer the following:

34a. How long did you work for that employer? Years Months

FIGURE 10.4 California's online unemployment form asks for data it could calculate itself in both of these questions.

Many factors affect learning

- Various factors affect how quickly people can learn to use interactive systems so proficiently that operating the system is handled largely by **automatic** cognitive processes
- Such factors include
  - Frequency and regularity of practice
  - Task-relatedness and familiarity of required actions and vocabulary
  - Consistency of operation
  - Riskiness of exploring

Many factors affect learning

- We learn faster when:
  - Practice is frequent, regular, and precise
  - Operation is task focused, simple, and consistent
  - Vocabulary is task focused, familiar, and consistent
  - Risk is low

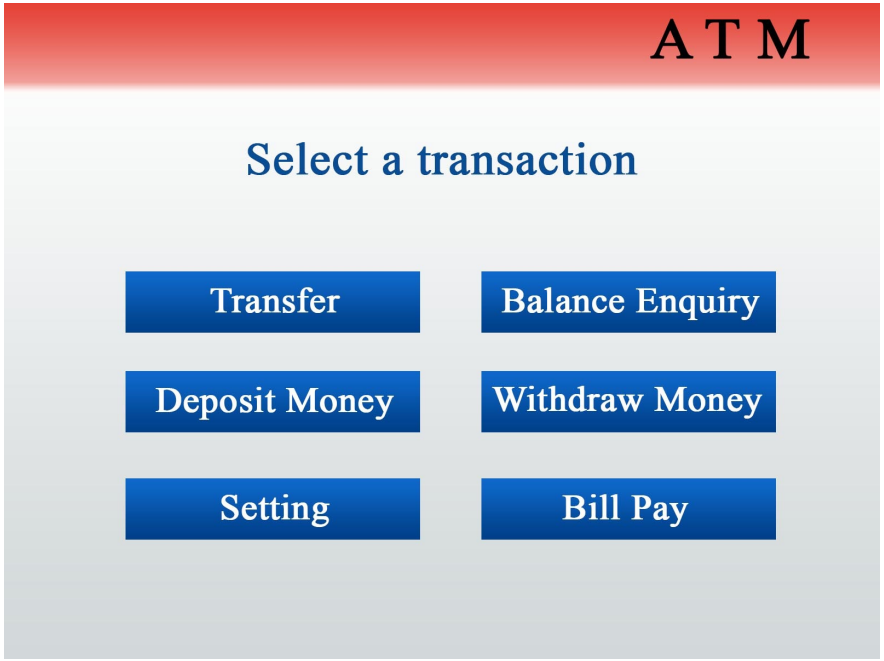
- Questions that designers should pose to themselves: **how can we design systems so that using them becomes automatic within a reasonable amount of time?**

We learn faster when practice is frequent, regular, and precise

- **Frequency of practice**
  - If people use an interactive system only rarely it is hard for them to remember from one time to the next the details of how to use it
  - If they use an interactive system frequently, familiarity develops quickly
  - Most user-interface designers are well aware of this, and therefore **design systems differently depending on whether people will use them casually and rarely or intensively and often**

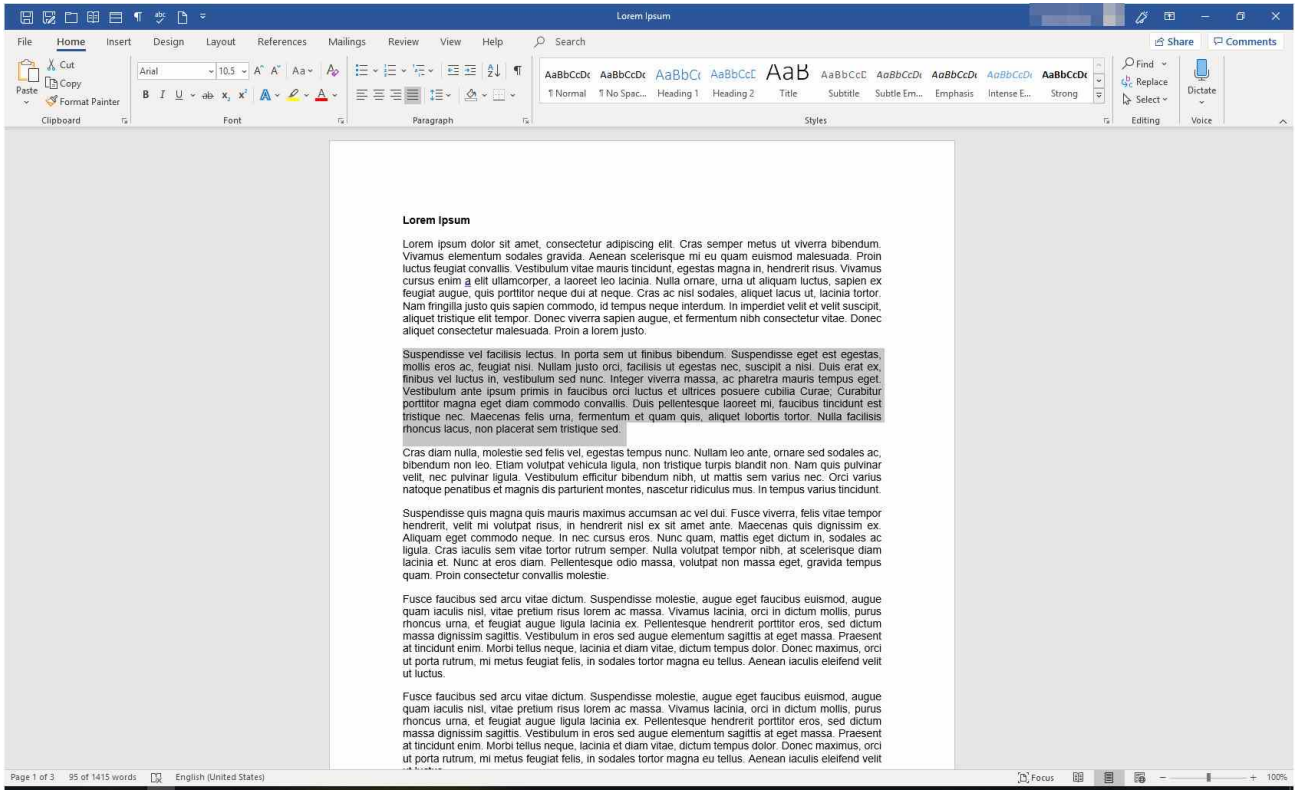
Frequency of practice: examples

- E.g., ATMs are designed under the assumption that people will not remember much from one usage to the next. They are designed to be simple and to remind people what they do and how they work. ATMs present a short list of anticipated user goals (e.g., withdraw cash, deposit funds, transfer funds), then guide users through the steps of the selected task



Frequency of practice: examples

- Other systems (such as document editing applications, air-traffic control systems, or online accounting services) are designed with the assumption that people will be using them daily – perhaps even minute-by-minute – and will quickly learn and remember details about how to use them

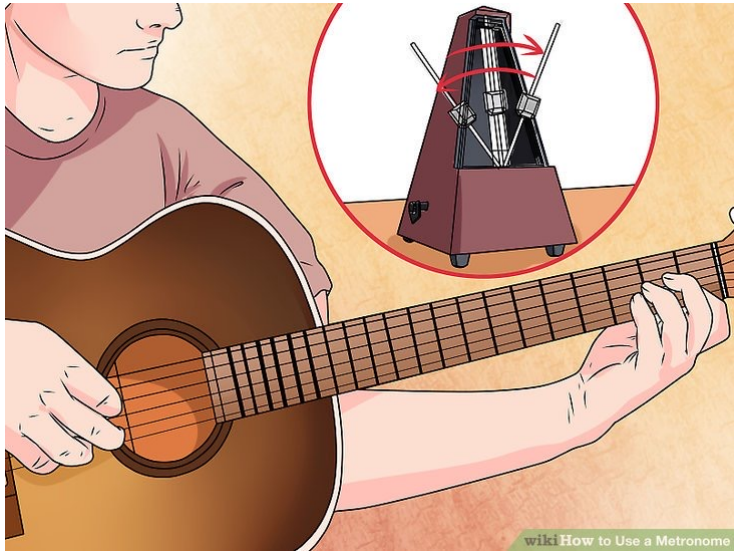


Regularity of practice

- How long does it take for an activity to become an automatic habit?
  - Studies have been conducted to measure this
  - Results show that forming automatic habits takes from 18 to 254 days, with more complex activities taking more time
  - Results also show that habits formed faster if practiced regularly (e.g., daily). Skipping a practice session now and then didn't make much difference, but skipping a lot of practice sessions significantly slowed participants' progress toward habit formation
- **Guideline**: if you want the use of your software to become habitual and automatic for user design it so as to encourage people to use it regularly

Precision of practice

- **The more carefully and precisely a person practices an activity, the better the brain learns**
- Practicing the same activity imprecisely just strengthens the imprecision
- To train your brain to make the activity precise, one must practice precisely and carefully, even if that requires practicing slowly at first or breaking down the activity into parts



- **Guideline:** if efficiency and precision are important for a task, design the supporting software and its documentation to:
  - **help people be precise** (e.g., by providing guides and grids)
  - **encourage people to use it purposefully and carefully** rather than distractedly and sloppily

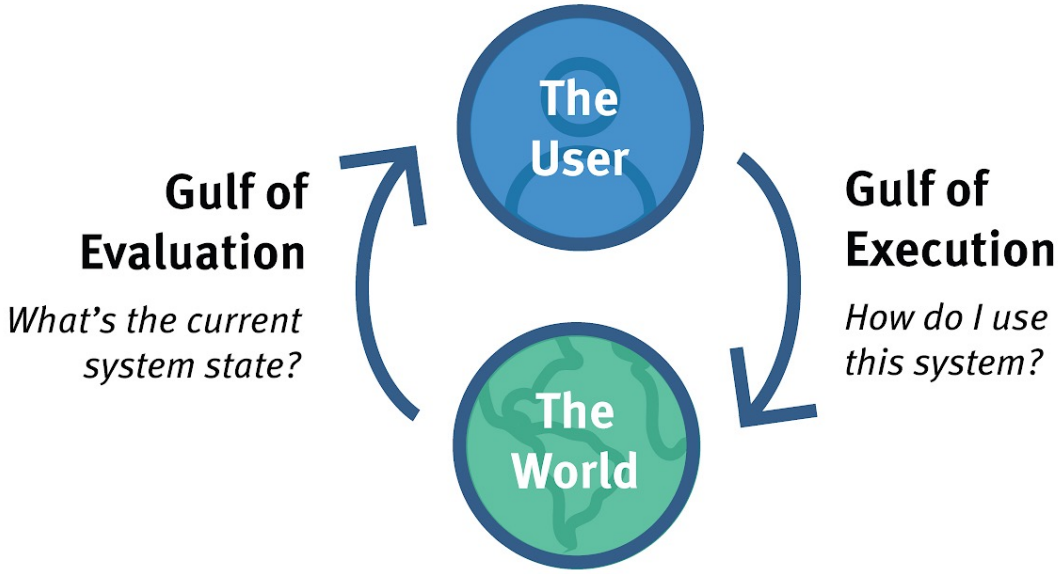
We learn faster when operation is task focused, simple, and consistent

- The motivation for this statement is rooted in the following concepts introduced in the next slides
  - **The two UX gulfs**
  - **Task analysis**
  - **Conceptual models**
  - **Consistency**

The two UX gulfs

- To successfully interact with technology, users must overcome two challenges
  - **Evaluation: Understanding the state of the system**
  - **Execution: Taking action to accomplish a specific goal**

- Cognitive psychologists call the gap between what a user wants and the operations the tool provides **"the gulf of execution"**
- The gap between the operations the tool provides and what the user wants is **"the gulf of evaluation"**



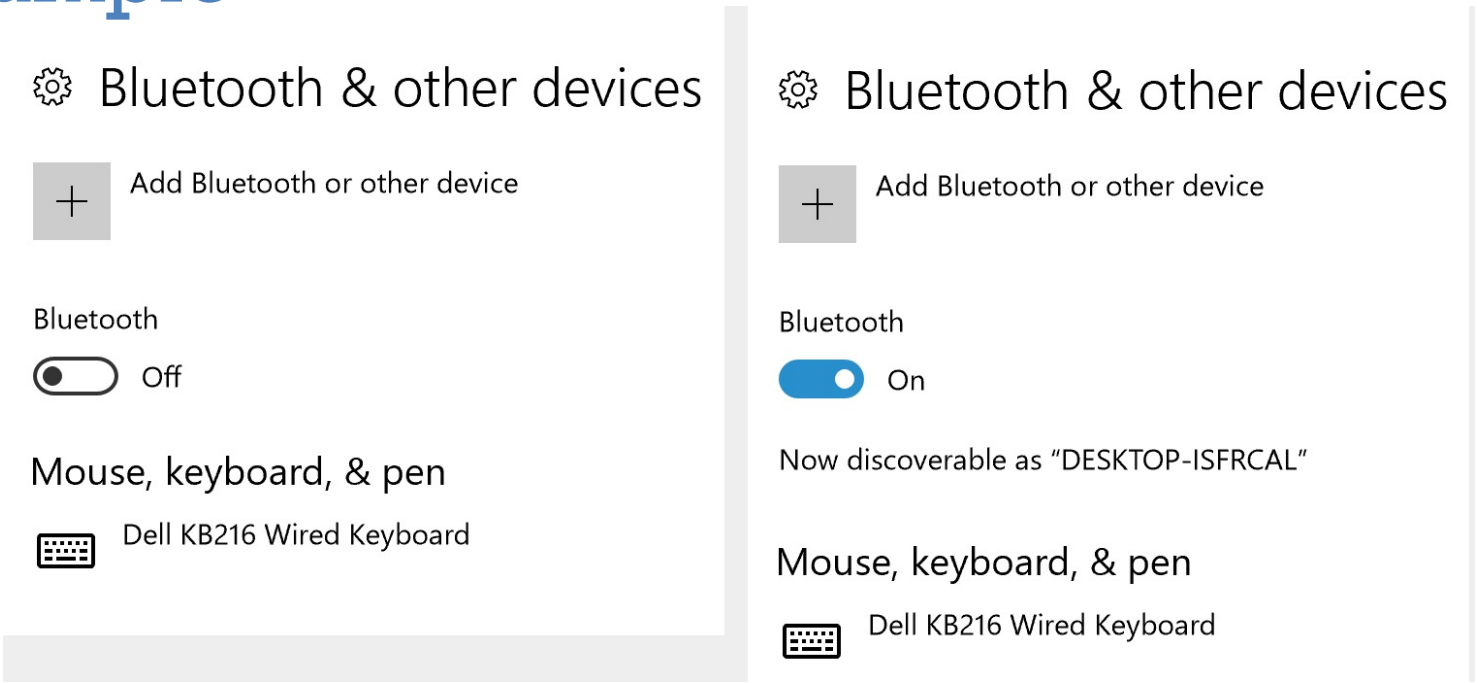
The two UX gulfs

- Users are in a cycle, alternating these two issues:
  - **after you execute, you must evaluate the outcome, plan to execute your next step, evaluate that outcome, and so on, until you reach your desired end-goal and quit**
- Both of these challenges are composed of **subtasks** which users must accomplish
  - Successful evaluation requires not just **perceiving** the system-status indicator, but also **interpreting** what it means
  - Execution requires both **planning** an action based on an understanding of how the controls work, and actually **manipulating** the controls

The two UX gulfs

- This type of granular, specific analysis of the interaction is important because success at one subtask doesn't necessarily mean success at the others
- The granularity of breaking down evaluation and execution into specific subtasks **sheds light on the detailed reasons why designs fail**

Example



- This Bluetooth-connectivity switch has a circle that can be shifted from side to side to change the state to Off (left) or On (right)
- The label reflects the current state of the control, not what will happen when the switch will be moved to the right (unlike the label for the plus icon, which explains what will happen when clicking it)
- A person looking at the left image could think that moving the circle to the right would set the Bluetooth to Off, instead the UI works in the opposite way: it would put it to On

The two UX gulfs

- A person using a tool must expend cognitive effort to translate what he or she wants into a plan based on the tool's available operations and vice versa
  - That cognitive effort pulls the person's attention away from the task and refocuses it on the tool
- The **smaller** the gulf between the operations that a tool provides and what its users want to do, the less the users need to think about the tool and the more they can concentrate on their task
  - As a result, **the tool becomes automatic more quickly**
- **The way to reduce the gulfs is to design tools providing operations that match what users are trying to do**

The two UX gulfs

- Designers can support the users by being aware of these gulfs and bridging them with a transparent conceptual model
- To design software, services, and appliances to provide operations matching users' goals and tasks, **designers must thoroughly understand the user goals and tasks the tool is intended to support**
- Gaining that understanding requires 3 steps:
  1. **Perform a task analysis**
  2. **Design a task-focused conceptual model, consisting mainly of an objects/actions analysis**
  3. **Design a user interface based strictly on the task analysis and conceptual model**

Task analysis: questions

- What goals do users want to achieve by using the application?
- What set of human tasks is the application intended to support?
- Which tasks are common, and which ones are rare?
- Which tasks are most important, and which ones are least important?
- What are the steps of each task?
- What are the result and output of each task?
- Where does the information for each task come from?
- How is the information that results from each task used?

Task analysis: questions

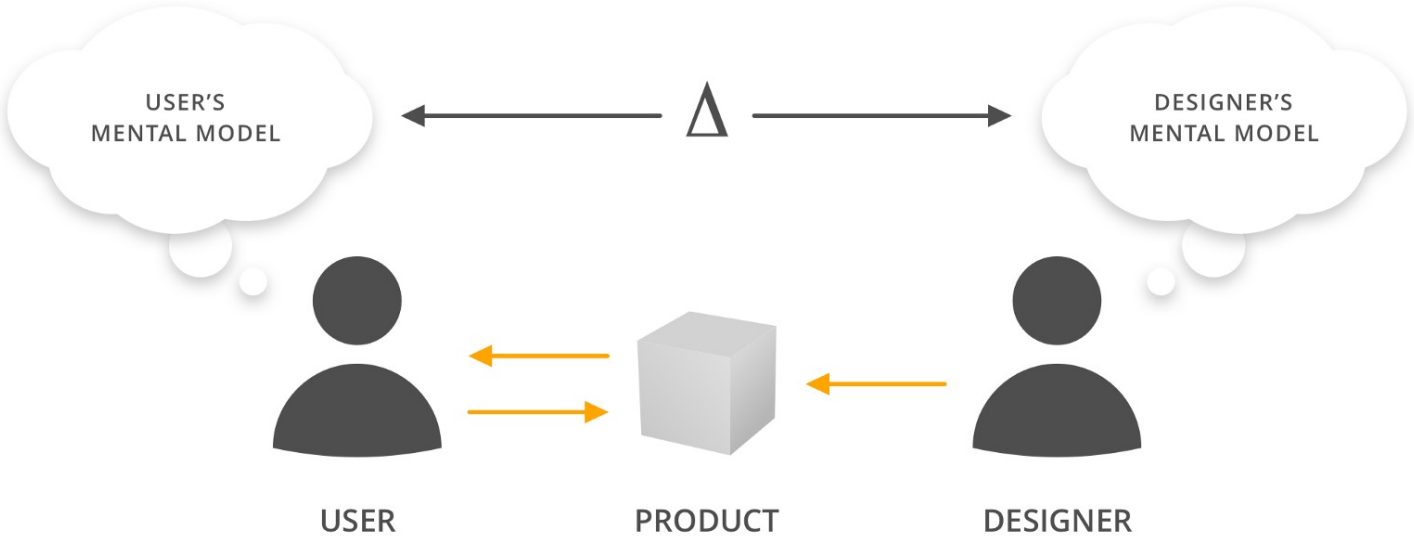
- Which people do which tasks?
- What tools are used to do each task?
- What problems do people have performing each task? What sorts of mistakes are common? What causes them? How damaging are mistakes?
- What terminology do people who do these tasks use?
- What communication with other people is required to do the tasks?
- How are different tasks related?

Task analysis: questions

- Once these questions are answered (by observing and/or interviewing people who do the tasks that the tool supports), the next step is **not** to start sketching possible user interfaces
- The next step is to **design a conceptual model** for the tool that focuses on the users' tasks and goals

Conceptual model

- A conceptual model of an application **is the model that the designers want users to understand**
- By using the application, talking with other users, and reading the documentation, users build a model in their minds of how to use it
- Hopefully, the model that users build in their minds is close to the one the designers intended; that is more likely when designers explicitly design a clear conceptual model as a key part of their development process




Developer: Makes a simple, intuitive UI
Users

Conceptual model

- A conceptual model is a theory of how a system works, what its signals mean, and what the outcomes of different user actions will be
- A conceptual model describes **abstractly** what tasks a user can perform with the system and what concepts they must be aware of to complete them
- The concepts should be those that came out of the task analysis, so the conceptual model is focused on the task domain

Conceptual model

- It should include **a few, if any, concepts for users to "master" that are not in the target task domain**
  - The more direct the mapping between the application's concepts and those of the tasks it is intended to support, the less translating users will have to do, and the easier the tool will be to learn
- In addition to being focused on users' tasks, a conceptual model should be as simple as possible
  - **Simpler means fewer** concepts: the fewer concepts a model has for users to master, the better, as long as it provides the required functionality
  - **"Less is more"**, as long as what is there fits well with users' goals and tasks

Conceptual model and feature requests

- In most development efforts, there is pressure to add extra functionality in case a user might want it
- **Resist** such pressure unless there is considerable evidence that a significant number of potential customers and users really need the extra functionality
- Why? **Because every extra concept increases the complexity of the software**. It is one more thing users have to learn
- But actually it is not just one more thing. Each concept in an application interacts with most of the other concepts, and those interactions result in more **complexity**

Conceptual model

- Interpretation requires effort, and most people try to minimize this effort by relying on a conceptual model to understand a system
- To save time, most people rely on their past experiences to quickly build mental models for new systems
- **Designers can leverage this natural tendency** by deliberately including design elements that help users build effective mental models. This requires:
  - Identifying a relevant design which users are already familiar with
  - Creating **visual similarities** between the new design and the familiar design
  - Creating **functional similarities** between the new design and the familiar design

Consistency

- The consistency of an interactive system strongly affects **how quickly its users progress from controlled, consciously monitored, slow operation to automatic, unmonitored, faster operation**
- The **more consistent** the system, the **more predictable** its function are
  - In a highly consistent system, the operation of a particular function is predictable from its type, so people quickly learn how everything in the system works and its use quickly becomes habitual
  - In an inconsistent system, users cannot predict how its different functions work, so they must learn each one anew, which slows their learning of the overall system and keeps their use of it a controlled, attention-consuming process

Consistency: 2 levels

- Consistency at **conceptual level**
  - is determined by the mapping between the objects, actions, and attributes of the conceptual model
  - Do most objects in the system have the same actions and attributes, or not?
- Consistency at **physical level**
  - It is determined by the mapping between the conceptual actions and the physical movements required to execute them
  - Are all conceptual actions of a certain type initiated and controlled by the **same physical movements**, or not?
  - We have a "muscle memory"
  - A common way that developers promote keystroke-level consistency is to **follow** look-and-feel **standards**

We learn faster when vocabulary is task-focused, familiar, and consistent

- Ensuring that a system exposes a small, consistent, and task-appropriate set of concepts to its users is a big first step, but it is not enough to minimize the time it takes for people to learn an interactive system
- You also have to make sure that the vocabulary – what concepts are called – fits the task, is familiar, and is consistent
  - **Terminology should be task-focused**
  - **Terminology should be familiar**
  - **Terminology should be consistent**

Terminology should be task-focused

- Usually, task-focused terms for concepts **emerge from the interviews and observations** of users that designers conduct as part of the task analysis
- Occasionally, software needs to expose a concept that is new to users; the challenge for a designer is keeping such concepts and their names focused on the task, **not on the technology**





FIGURE 11.2 iCasualties.org uses language that is not task focused ("database") in its instructions.

Visitors to this site don't care or need to know that the Web site's data is stored in multiple databases. Task-focused instructions would ask them to select a "country", not a "database"

Terminology should be familiar

- **To reduce the time** it takes for people to master your system (so that using it becomes automatic) **don't force them to learn a whole new vocabulary**
- Familiar words are easier to read and understand because they can be recognized automatically
- Unfamiliar words cause people to use more conscious decoding methods, which consumes the scarce resources of the short-term memory and thereby lowers comprehension

Terminology should be consistent

- The terminology used in an interactive system is consistent when **each concept has one and only one name**
- **Never use different terms for the same concept, or the same term for different concepts**
- Even terms that are ambiguous in the real world should mean only one thing in the system. Otherwise, the system will be harder to learn and remember

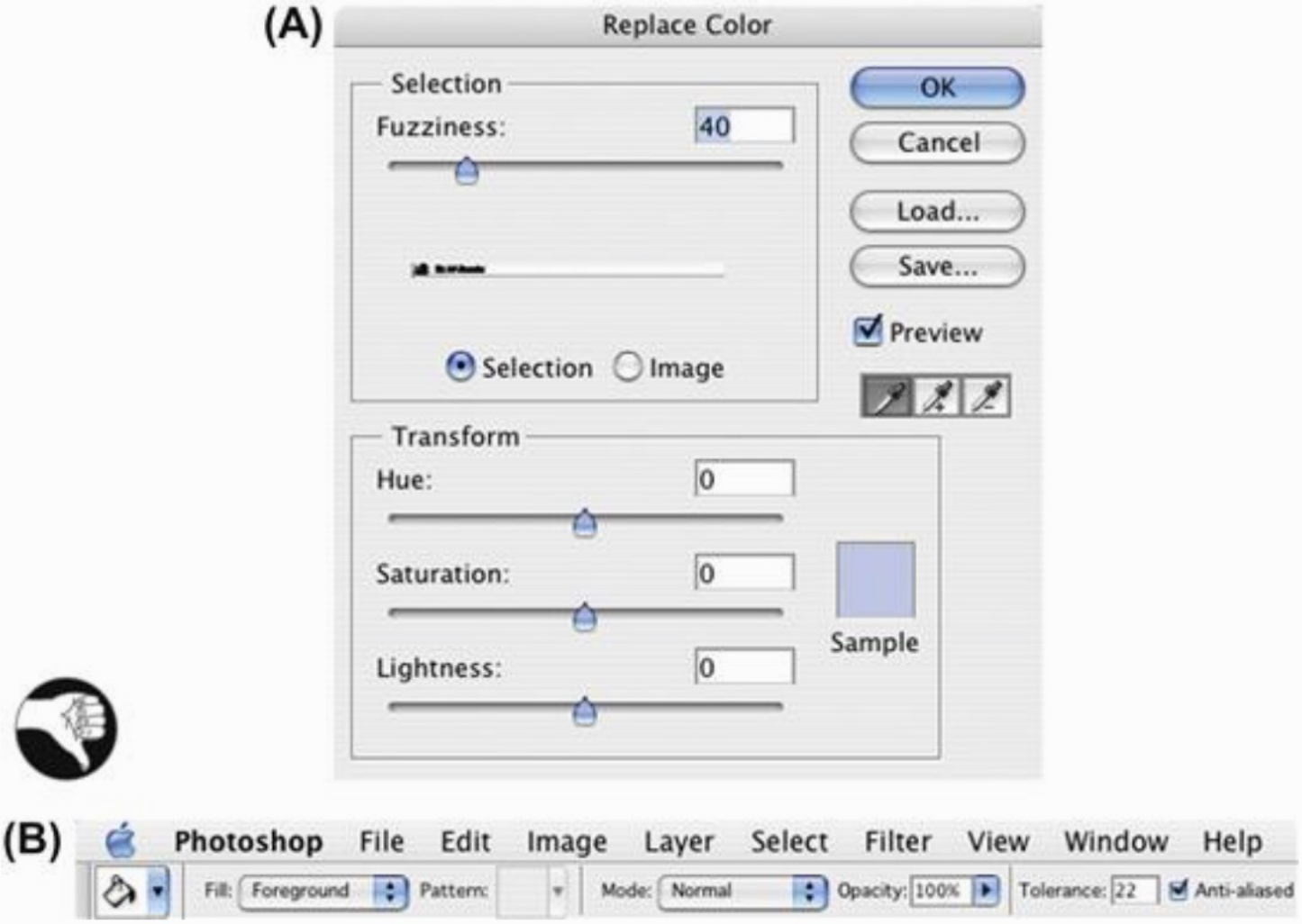


FIGURE 11.8 Photoshop uses different names for the tolerance parameter in two color-replacement functions: (A) "Fuzziness" in Replace Color, and (B) "Tolerance" in Paint Bucket.

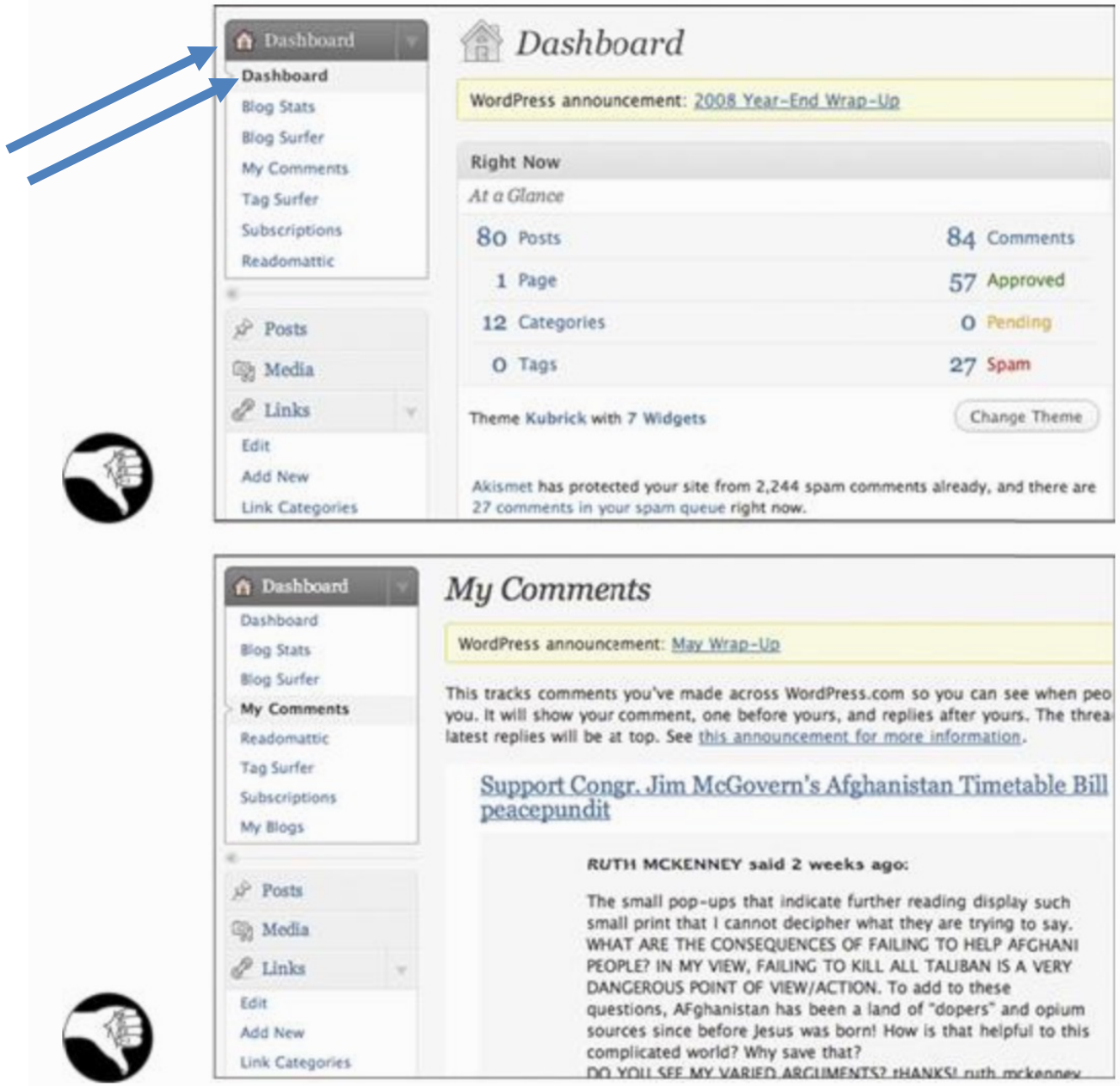


FIGURE 11.9 At WordPress.com, "Dashboard" means both a blog's entire administrative area and a certain page in it.

When risk is low, we explore more and we learn more

- Most interactive systems have far more functionality than most of their users ever try
  - Often people don't even know about most of the functionality provided by software or gadgets they use every day
  - One reason for this is fear of doing mistakes
- People make mistakes. Many interactive systems make it too easy for users to make mistakes, do not allow users to correct mistakes, or make it costly or time consuming to correct mistakes
  - People won't be very productive in using such systems: they will waste too much time correcting or recovering from mistakes

When risk is low, we explore more and we learn more

- Even more important than the impact on time is the impact on practice and exploration
- A high-risk system, in which mistakes are easy to make and costly, discourages users
- Creating a low-risk environment means doing the following:
  - **Prevent errors where possible**
  - **Deactivate invalid commands**
  - **Make errors easy to detect by showing users clearly what they have done** (e.g., deleting a paragraph by mistake)
  - **Allow users to undo, reverse, or correct errors easily**

Design

- After you have designed a conceptual model that is **task focused**, as **simple** as possible, and as **consistent** as possible, you can design a user interface for it that **minimizes** the time and experience required for using the application to become automatic

Lesson for the course “Human-Computer Interaction”

Axure RP (Part 6)

Prof. Luca Turchet

UNIVERSITY OF TRENTO - Italy

Repeaters

- See the documentation page:
  - https://docs.axure.com/axure-rp/widgets/repeaters/
- Tutorial:
  - https://docs.axure.com/tutorials/repeaters/adding-rows/
  - https://docs.axure.com/tutorials/repeaters/deleting-rows/
  - https://docs.axure.com/tutorials/repeaters/updating-rows/
  - https://docs.axure.com/tutorials/repeaters/marking-rows/
  - https://docs.axure.com/tutorials/repeaters/sorting-rows/
  - https://docs.axure.com/tutorials/repeaters/filtering-rows/
  - https://docs.axure.com/tutorials/repeaters/pagination/

Homework

- Choose an interactive systems and assess whether its use complies with the guidelines listed in slides n. 8, 9, 10, 11 and 12
- Choose an interactive systems and assess whether there are aspects of it that distract users by imposing technical problems and goals that users don't want
- Choose an interactive system and assess its consistency
- Choose an interactive system and assess its terminology
- Choose an interactive system and assess its risk level for doing mistakes
- Perform the task analysis for the interactive system of your project and devise an accurate conceptual model

Bibliography

- Chapters 10, and 11 of the book *"Designing with the Mind in Mind: Simple Guide to Understanding User Interface Design Guidelines"* - 2nd Edition - by Jeff Johnson
- https://www.nngroup.com/articles/two-ux-gulfs-evaluation-execution/